

REAL-TIME WITH PAUSE · RPG

# RTwP RPG

전투와 조작의 이원화

발표자 유창완



2026-06-21



[github.com/llabrynth/RTwP\\_RPG](https://github.com/llabrynth/RTwP_RPG)

# 목차

01  
비전  
Vision

02  
문제 정의  
The Problem

03  
개발 로드맵  
Roadmap

04  
구현 과정  
Implementation

05  
결과  
result

06  
Q & A  
Questions

# 전투는 긴박하지만 생각할 시간은 많습니다.

실시간의 긴장감과 턴제의 전략성을 동시에

# 두 장르의 장점만 결합한 게임이 없었다

실시간 게임(스타크래프트·격투)은 전체를 동시에 조작하기 버겁고 어렵습니다. 턴제는 템포가 느립니다. 둘의 장점만 합친 게임은 없었습니다.

| 턴제 RPG |      |     |
|--------|------|-----|
| 전략성    | ★★★★ |     |
| 템포     |      | 느림  |
| 파티 관리  |      | 완전  |
| 한계     |      | 기다림 |

| 이 게임  |      |            |
|-------|------|------------|
| RTwP  |      |            |
| 전략성   | ★★★★ |            |
| 템포    |      | 빠름         |
| 파티 관리 |      | 완전         |
| 차별점   |      | 완전한 조작과 속도 |

| 실시간 · RTS/격투 |     |       |
|--------------|-----|-------|
| 전략성          | ★★★ |       |
| 템포           |     | 빠름    |
| 파티 관리        |     | 어려움   |
| 한계           |     | 조작 부담 |

# 개발 로드맵 — 코어 먼저, 콘텐츠는 그 다음

|                                                                                                                                                   |                                                                                                                         |                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <div>1단계 · APR 05–09</div> <div>코어 전투 파이프라인</div> <div>Stage 0 · 기반 Stage 1 · 전투 기반</div> <div>자동 정지 엔진 · 20 TPS 틱 · 전투 판정 · 적 AI · 비주얼 피드백</div> | <div>2단계 · APR 11 – MAY 16</div> <div>콘텐츠 레이어</div> <div>Stage 기반 검증</div> <div>캐릭터 심화 · 월드 확장 · 스토리/진행 시스템 · 폴리싱</div> | <div>3단계 · MAY 22 – JUN 21</div> <div>품질 개선</div> <div>섹션 7 – 43</div> <div>고정 틱 마이그레이션 · UI 전면 개편(OT2) · 애니메이션 품질 · 버그 패치</div> |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|

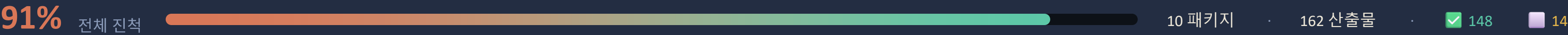
▶

마일스톤 1 — 04·09 코어 파이프라인 완성, 1스테이지 동작 검증 후 콘텐츠 확장



# WBS — 범위 기반 작업 분해

완료 기준 = 산출물 동작 + 통합 테스트 통과



# 숫자로 보는 과정

14

주 개발 기간

43

섹션 완료

17

ADR 설계 문서

5

실행 구성

# 구현 방법 — 20 TPS 고정 톱

```
// BattleSimulation.cs — 자동 정지 판정
private bool CheckAutoFreeze()
{
    return PartyManager.Instance.Members
        .Exists(m => m.Stats.IsAlive
            && !m.Sim.IsBusy);
}
```

- 살아있는 파티원 중 `IsBusy=false`가 한 명이라도 있으면 **정지**
- **20 TPS accumulator**로 판정·이동·AI를 동시 처리
- 렌더 프레임은 `Interpolate()`로 부드럽게 보간



# ADR 목록 — 17개 설계 결정

|          |                             |          |                         |
|----------|-----------------------------|----------|-------------------------|
| ADR-0001 | RTwP 전투 방식 채택 ★             | ADR-0010 | 코루틴 → 20 TPS 고정 틱 ★     |
| ADR-0002 | ScriptableObject 데이터 아키텍처 ★ | ADR-0011 | EventBus.Clear() 재구독 패턴 |
| ADR-0004 | 리플레이 별도 싹 분리                | ADR-0012 | _hadEnemies 플래그 설계      |
| ADR-0006 | StageConfig + WaveRunner    | ADR-0013 | 투사체 틱 기반 시뮬레이션          |
| ADR-0007 | ChapterHub 별도 싹             | ADR-0014 | Sorting Layer 계층 정리     |
| ADR-0008 | StageRunner.Update() 승패 폴링  | ADR-0015 | 스킬 트리 레이아웃              |
| ADR-0009 | Modern GDR + Cainos UI 방향   | ADR-0016 | 이펙트 레이어 분리              |

# ADR 심화

## ADR-0001 RTwP 전투 방식 채택

**Q** 왜 턴제나 액션 대신 RTwP인가?

**A** 전술적 포지셔닝(배후 +30% · 측면 +15%)과 실시간 긴장을 동시에. 스페이스바가 아니라 **파티원이 명령 대기**가 되면 **자동 정지** — 선딜 · 후딜 설계가 흐름을 결정.

## ADR-0010 코루틴 → 20 TPS 고정 틱

**Q** 왜 직접 틱 시스템을 만들었나?

**A** WaitForSeconds는 timeScale 의존인데 RTwP는 `IsPaused` 플래그로 제어 → 불일치. 20 TPS accumulator로 전환하니 정지 · 재개가 완벽 동기화.

## ADR-0002 ScriptableObject 데이터 아키텍처

**Q** 왜 데이터를 SO로 관리하나?

**A** 런타임 상태와 정적 데이터를 엄격 분리. Inspector 편집 · git diff 추적 용이. `GameAssets` 싱글턴 레지스트리로 **코드 수정 없이 콘텐츠만 추가**.

# 앱 구조 — 레이어 & 디렉토리

- Assets/Scripts/
- └─ Core/ 전역 싱글턴 · EventBus · SaveLoad
  - └─ Data/ ScriptableObject 정의
  - └─ Battle/ 시뮬·AI·판정 ★ 20 TPS
    - └─ Events/ EventBus 이벤트
    - └─ Conditions/ 상태이상
  - └─ UI/ 화면 · HUD · 패널
  - └─ Audio/ BGM 크로스페이드 · SFX 풀
  - └─ VFX/ 파티클 풀 · 상태이상
  - └─ Replay/ 전투 재생
  - └─ Tutorial/ 4단계 시퀀서

씬 흐름



# 기술 스택

|     |                                                                |    |                                                 |
|-----|----------------------------------------------------------------|----|-------------------------------------------------|
| 엔진  | <div>Unity 6 · URP 17.3</div> <div>2D/3D 혼합 경량 렌더</div>        | 언어 | <div>C#</div> <div>Unity 네이티브 · 강타입</div>       |
| 데이터 | <div>ScriptableObject</div> <div>Inspector 편집 · Git diff</div> | 저장 | <div>Newtonsoft JSON</div> <div>3슬롯 로컬 저장</div> |
| 입력  | <div>New Input System</div> <div>확장성</div>                     | AI | <div>AI NavMesh</div> <div>적 이동 경로</div>        |
| 테스트 | <div>Unity Test Framework</div> <div>유닛 · 통합</div>             | 통신 | <div>EventBus (정적)</div> <div>느슨한 결합</div>      |

# 적용 기술 — AI 페어 개발 도구체인

Claude Code 스킬 · Unity 연결

|    |                                                            |         |                                                                                                                   |
|----|------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------|
| 01 | <div>Superpowers</div> <div>obra/superpowers</div>         | 개발 방법론  | 브레인스토밍 → 계획 → TDD 구현 → 코드 리뷰 → 마무리를 자동 강제하는 오픈소스 스킬 프레임워크. 코드 전에 설계·테스트를 먼저 하도록 규율.                               |
| 02 | <div>Agent Skills</div> <div>addyosmani/agent-skills</div> | 스킬팩     | DEFINE → PLAN → BUILD → VERIFY → REVIEW → SHIP 개발 생애주기를 8개 슬래시 커맨드·24개 스킬로 인코딩. 스펙·테스트·리뷰·보안 등 시니어 워크플로를 일관되게 강제. |
| 03 | <div>Unity MCP</div> <div>Model Context Protocol</div>     | 에디터 브리지 | MCP로 AI를 Unity에디터에 직접 연결 . 씬 제어·에셋 관리·스크립트 편집·테스트 실행·프로파일링을 에디터 안에서 자동화 — AI 페어 개발의 통로.                           |
| 04 | <div>Graphify</div> <div>safishamsi/graphify</div>         | 지식 그래프  | 코드·문서·PDF·이미지를 읽어 지식 그래프를 구축 하는 멀티모달 Claude Code 스킬. 흩어진 자료의 숨은 구조·연결을 드러내고 쿼리당 토큰을 크게 절감.                        |



# 개발 환경 & 설치 가이드

## 환경

Unity 6 (6000.0.x) · URP 17.3.0 IDE Visual Studio 2022 / Rider OS Windows 11 / Claude Code

## 주요 패키지 manifest.json

inputsystem 1.18.0 · ai.navigation 2.0.13 timeline 1.8.12 · newtonsoft-json 3.2.2 test-framework 1.6.0 · recorder  
recorder 5.1.6

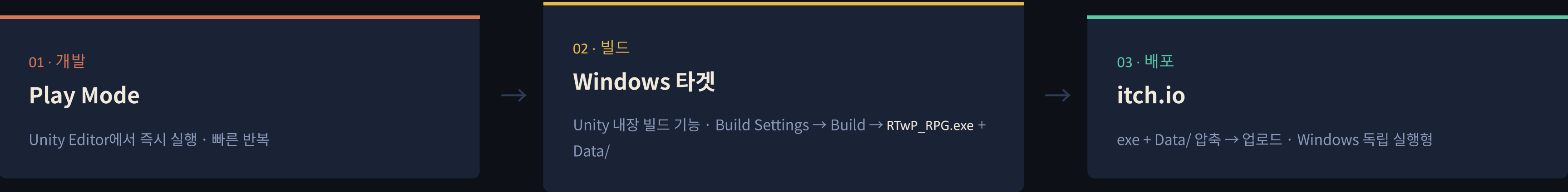
## 설치 & 실행

RTwP\_RPG 다운로드

→ RTwP\_RPG.zip 압축풀기

→ RTwP\_RPG.exe 실행

# 빌드 & 배포 파이프라인



# 아키텍처 레이어





# 시행착오 3선

## 코루틴 → 고정 틱

①

BEFORE

WaitForSeconds → sc-camel-time-scale=0 → UI/오디오 동결

AFTER

unscaledDeltaTime accumulator 20 TPS → 독립 제어 ✓

## Animator → TickSpriteAnimator

②

BEFORE

AnimatorUpdateMode.Manual → 정지되지 않는 애니메이션

AFTER

AnimationClip.SampleAnimation() 직접 호출 → 틱 동기화 ✓

## EventBus 리스너 누적

③

BEFORE

씬 전환 후 DontDestroyOnLoad 중복 구독 → 이벤트 N배 발행

AFTER

FadeAndLoad() → EventBus.Clear() → 재구독 패턴 ✓

# 성능 최적화

| 항목          | 방법                                          | 효과          |
|-------------|---------------------------------------------|-------------|
| GC alloc 방지 | Queue<T> 오브젝트 풀 · DamagePopup·VFX·스냅샷       | GC 스파이크 제거  |
| AI 틱 최적화    | EnemyGroupCoordinator — 그룹 전략 일괄 계산         | 유닛별 비용 절감   |
| 애니메이션       | Animator off · SampleAnimation() SimTick 단위 | CPU 절감      |
| 정렬 캐싱       | InitiativeQueue._dirty — 변화 시에만 재정렬         | 매 프레임 정렬 제거 |
| 씬 로드        | ScreenFader 페이드 중 씬 로드                      | 체감 로딩 개선    |

# 코드 품질 관리 — AI 도구로 강제한 규율

## 품질 관리 원칙

- ADR 17개 설계 사전 문서화 · WBS 자동 갱신 · 주간 보고서 docs/weekly/ ·
- 인터페이스 기반 설계 · 단일 책임 분리

## AI 도구로 유지한 품질

**Superpowers**    코드 작성 전 설계·TDD를 강제해 결함을 사전 차단

**Agent Skills**    코드 리뷰·보안·단순화 등 시니어 품질 게이트를 워크플로로 강제

**Unity MCP**    에디터에서 컴파일·검증을 즉시 실행하고 오류 자가 수정 루프

**Graphify**    코드·문서를 지식 그래프로 연결해 구조·의존성을 일관되게 파악

🔧 **지속적 개선** — 주간 보고서·WBS 자동 갱신·ADR 지속 작성으로 설계를 계속 다듬고, AI 도구 워크플로로 리뷰·리팩토링을 반복 적용 중

# 단위 테스트 결과

Unity Test Framework 1.6.0 · EditMode · NUnit

|                                                                |                                |                                  |                                 |                                     |
|----------------------------------------------------------------|--------------------------------|----------------------------------|---------------------------------|-------------------------------------|
| <div>24 /24</div> <div>PASSED</div> <div>실패 0 · 통과율 100%</div> | <div>4</div> <div>테스트 파일</div> | <div>24</div> <div>테스트 케이스</div> | <div>7</div> <div>자동화 시스템</div> | <div>5</div> <div>수동 통합 검증 항목</div> |
|----------------------------------------------------------------|--------------------------------|----------------------------------|---------------------------------|-------------------------------------|

|                                                                                                                                               |                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <div>PositioningTests.cs</div> <div>✓ 6 PASSED</div> <div>대상 PositioningCalculator · 포지셔닝 판정</div> <div>배후 ×1.4 · 측면 × 1.2 · 정면 ×1.0</div>    | <div>CombatCalculatorTests.cs</div> <div>✓ 2 PASSED</div> <div>대상 CombatCalculator · 데미지 계산</div> <div>20×1.4 = 28 · 20×1.2 = 24</div> |
| <div>Stage2LogicTests.cs</div> <div>✓ 9 PASSED</div> <div>대상 ClassData · SkillTree · StatusEffect</div> <div>전직 조건 3 · 스킬 해금 4 · 상태이상 2</div> | <div>Stage3LogicTests.cs</div> <div>✓ 7 PASSED</div> <div>대상 DifficultyScaler · ChapterProgress</div> <div>레벨 스케일링 4 · 단서 게이지 3</div>  |

**커버리지** 순수 로직(수치 계산·데이터 유효성)은 자동화 24개로 커버 · RTwP 자동 정지 타이밍·씬 전환·리플레이·튜토리얼·빌드는 수동 플레이 검증

# 활용 방안

## AI 엔지니어링 역량

복잡한 전투 디버깅 · AI 에이전트 엔지니어링 경험을 다른 프로젝트에도 그대로 적용



# Q & A